Double Ratchet Algorithm: Full Code Notes

1. Imports Section

```
import os
import base64
import logging
from typing import Optional, Tuple, Dict, Any

from cryptography.hazmat.primitives.asymmetric import x25519
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives import hashes, hmac
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives import serialization
from cryptography.exceptions import InvalidTag

logger = logging.getLogger("double_ratchet")
```

Used for:

- `os` : Random bytes generation.
- `base64` : Convert bytes to JSON-safe strings.
- `logging` : Debug messages.
- `x25519` : Diffie-Hellman Key Exchange. Signal protocol uses Curve25519 / X25519 to generate shared secrets.
- `HKDF` : Key Derivation Function. Converts: Old Root Key + DH output into New Root Key & New Chain Key.
- `hashes, hmac` : HMAC-SHA256 to generate: Message Keys, Next Chain Keys.
- `AESGCM` : Message Encryption. AES-GCM gives: ✔ Confidentiality ✔ Integrity ✔ Authentication.

2. Constants

```
MAX_SKIP = 1000

NONCE_LEN = 12   # AES-GCM standard nonce length
KEY_LEN = 32     # 256-bit keys throughout (RK, CK, MK all 32 bytes)
```

Used for:

- `MAX_SKIP = 1000` : Purpose: Prevent memory attacks. Example: Suppose attacker sends: `message_number = 1000000`. Program would try storing a million skipped keys. This prevents that.
- `NONCE_LEN = 12` : AES-GCM standard nonce size.
- `KEY_LEN = 32` : 32 bytes (256 bits). Used for: Root Key, Chain Key, Message Key.

3. Exception Classes

```
class DoubleRatchetError(Exception):
    """Base class for all errors raised by this module."""

class DecryptionError(DoubleRatchetError):
    """Raised when a message fails to decrypt/authenticate."""

class TooManySkippedMessagesError(DoubleRatchetError):
    """Raised when a peer's message implies skipping more than MAX_SKIP keys."""
```

Used for:

- `DoubleRatchetError`: Base error.
- `DecryptionError`: Raised when: Wrong key, Tampered message, or Modified ciphertext.
- `TooManySkippedMessagesError`: Raised when: Skipped messages > 1000.

4. Serialization Helpers

```
def bytes_to_b64(data: bytes) -> str:
    return base64.b64encode(data).decode("ascii")

def b64_to_bytes(data: str) -> bytes:
    return base64.b64decode(data)

def header_to_dict(header: Dict[str, Any]) -> Dict[str, Any]:
    return {
        "dh_pub": bytes_to_b64(header["dh_pub"]),
        "pn": header["pn"],
        "n": header["n"],
    }

def header_from_dict(data: Dict[str, Any]) -> Dict[str, Any]:
    return {
        "dh_pub": b64_to_bytes(data["dh_pub"]),
        "pn": int(data["pn"]),
        "n": int(data["n"]),
    }
```

Used for:

These functions are for the server.

- `bytes_to_b64()`: Converts `b' 4'` to `"EjQ="`. Reason: JSON cannot directly send bytes.
- `b64_to_bytes()`: Reverse operation.
- `header_to_dict()`: Converts `{'dh_pub': bytes, 'pn': int, 'n': int}` into JSON-safe data.
- `header_from_dict()`: Restores it back.

5. Key Serialization

```
def public_key_to_bytes(public_key: x25519.X25519PublicKey) -> bytes:
    return public_key.public_bytes(
        encoding=serialization.Encoding.Raw,
        format=serialization.PublicFormat.Raw,
    )

def public_key_from_bytes(data: bytes) -> x25519.X25519PublicKey:
    return x25519.X25519PublicKey.from_public_bytes(data)
```

Used for:

- `public_key_to_bytes()`: Converts `X25519PublicKey` into 32-byte raw format. Needed for network transmission.
- `public_key_from_bytes()`: Opposite operation.

6. Main Class & Initialization

```
class DoubleRatchet:
    def __init__(self, shared_secret: bytes, dh_key: Any, is_bob: bool = False):
        self.MKSKIPPED: Dict[Tuple[bytes, int], bytes] = {}
        self.PN = 0
        self.Ns = 0
        self.Nr = 0

        if is_bob:
            self.DHs = dh_key
            self.DHr: Optional[x25519.X25519PublicKey] = None
            self.RK = shared_secret
            self.CKs: Optional[bytes] = None
            self.CKr: Optional[bytes] = None
        else:
            self.DHs = self._generate_dh()
            self.DHr = dh_key
            dh_out = self._dh(self.DHs, self.DHr)
            self.RK, self.CKs = self._kdf_rk(shared_secret, dh_out)
            self.CKr = None
```

Used for:

This is the heart of the algorithm. It stores: RK = Root Key, CKs = Sending Chain Key, CKr = Receiving Chain Key, DHs = My DH key pair, DHr = Other person's DH public key, Ns = Sent messages count, Nr = Received messages count, PN = Previous chain length.

This starts the ratchet after X3DH.

- **Alice Side:** `self.DHs = self._generate_dh()` (Alice creates new ratchet key). `dh_out = self._dh(self.DHs, self.DHr)` (Performs ECDH). `self.RK, self.CKs = self._kdf_rk(...)` (Produces Root Key, Sending Chain Key). Alice can immediately send messages.
- **Bob Side:** Bob starts with `CKs = None`, `CKr = None` because Bob hasn't received Alice's first ratchet key yet.

7. DH Key Generation & Exchange

```
    @staticmethod
    def _generate_dh() -> x25519.X25519PrivateKey:
        return x25519.X25519PrivateKey.generate()

    @staticmethod
    def _dh(private_key: x25519.X25519PrivateKey, public_key: x25519.X25519PublicKey) -> bytes:
        return private_key.exchange(public_key)
```

Used for:

- `_generate_dh()` : `X25519PrivateKey.generate()` Creates a fresh ratchet key pair. Every DH ratchet generates a new one.
- `_dh()` : `private.exchange(public)` Performs ECDH. Result: Shared Secret.

8. Key Derivation Functions (KDF)

```
    @staticmethod
    def _kdf_rk(rk: bytes, dh_out: bytes) -> Tuple[bytes, bytes]:
        hkdf = HKDF(algorithm=hashes.SHA256(), length=2 * KEY_LEN, salt=rk, info=b"DoubleRatchet-
RootChain")
        derived = hkdf.derive(dh_out)
        return derived[:KEY_LEN], derived[KEY_LEN:]

    @staticmethod
    def _kdf_ck(ck: bytes) -> Tuple[bytes, bytes]:
        h_mk = hmac.HMAC(ck, hashes.SHA256())
        h_mk.update(b"\x01")
        message_key = h_mk.finalize()

        h_ck = hmac.HMAC(ck, hashes.SHA256())
        h_ck.update(b"\x02")
        next_chain_key = h_ck.finalize()

        return next_chain_key, message_key
```

Used for:

- `_kdf_rk(rk, dh_out)` : Most important function. Input: Old Root Key, New DH Output. HKDF generates: New Root Key, New Chain Key. Why? Forward secrecy.
- `_kdf_ck(ck)` : Second most important function. Input: Current Chain Key. Produces: Next Chain Key, Message Key. Using HMAC. This is the symmetric ratchet (Flow: CK0 -> MK0+CK1 -> MK1+CK2 -> MK2+CK3).

9. Skipped Message Handling

```
    def _try_skipped_message_keys(self, header: Dict[str, Any]) -> Optional[bytes]:
        key = (header["dh_pub"], header["n"])
        if key in self.MKSKIPPED:
            return self.MKSKIPPED.pop(key)
        return None

    def _skip_message_keys(self, until: int) -> None:
        if self.CKr is not None:
            dh_pub_bytes = public_key_to_bytes(self.DHr)
            while self.Nr < until:
                self.CKr, mk = self._kdf_ck(self.CKr)
                self.MKSKIPPED[(dh_pub_bytes, self.Nr)] = mk
                self.Nr += 1
```

Used for:

- `_try_skipped_message_keys()` : Suppose Message 5 arrives before Message 4. Program stores Message 4 key. Later it retrieves it.
- `_skip_message_keys()` : Generates all missing keys. Example: Current `Nr = 2`, Incoming `n = 5`. Generates keys for 3 and 4, stores them. Then decrypts 5.

10. DH Ratchet

```
    def _dh_ratchet(self, header: Dict[str, Any]) -> None:
        self.PN = self.Ns
        self.Ns = 0
        self.Nr = 0

        self.DHr = public_key_from_bytes(header["dh_pub"])

        dh_out_receive = self._dh(self.DHs, self.DHr)
        self.RK, self.CKr = self._kdf_rk(self.RK, dh_out_receive)

        self.DHs = self._generate_dh()
        dh_out_send = self._dh(self.DHs, self.DHr)
        self.RK, self.CKs = self._kdf_rk(self.RK, dh_out_send)
```

Used for:

Most important Signal feature. Triggered when `header["dh_pub"]` changes. Meaning: Peer generated new DH key.

Steps:

1. `self.DHr = new public key`
2. `DH(old private, new public)` (Creates New Receiving Chain)
3. Generate new local key pair: `self.DHs = self._generate_dh()`
4. `DH(new private, new public)` (Creates New Sending Chain)

Result: New Root Key, New Sending Chain, New Receiving Chain. This gives Forward Secrecy & Post-Compromise Security.

11. Encrypt API

```
    def encrypt(self, plaintext: bytes, associated_data: bytes = b"") -> Tuple[Dict[str, Any], bytes]:
        self.CKs, mk = self._kdf_ck(self.CKs)

        header = {
            "dh_pub": public_key_to_bytes(self.DHs.public_key()),
            "pn": self.PN,
            "n": self.Ns,
        }
        self.Ns += 1

        ad_combined = self._build_associated_data(header, associated_data)

        aesgcm = AESGCM(mk)
        nonce = b"\x00" * NONCE_LEN
        ciphertext = aesgcm.encrypt(nonce, plaintext, ad_combined)

        return header, ciphertext
```

Used for:

Workflow:

1. Generate: `self.CKs, mk = self._kdf_ck(self.CKs)` (Creates Message Key).
2. Create header: `{ dh_pub, pn, n }`
3. Build Associated Data: `_build_associated_data()` (Protects header from modification).
4. Encrypt: `AESGCM(mk)` (Produces Ciphertext).

12. Decrypt API

```
    def decrypt(self, header: Dict[str, Any], ciphertext: bytes, associated_data: bytes = b"") ->
bytes:
        mk = self._try_skipped_message_keys(header)

        if mk is None:
            current_dhr_bytes = public_key_to_bytes(self.DHr) if self.DHr else None
            if header["dh_pub"] != current_dhr_bytes:
                self._skip_message_keys(header["pn"])
                self._dh_ratchet(header)

            self._skip_message_keys(header["n"])
            self.CKr, mk = self._kdf_ck(self.CKr)
            self.Nr += 1

        nonce = b"\x00" * NONCE_LEN
        ad_combined = self._build_associated_data(header, associated_data)
        aesgcm = AESGCM(mk)

        try:
            plaintext = aesgcm.decrypt(nonce, ciphertext, ad_combined)
        except InvalidTag as exc:
            raise DecryptionError("Message failed authentication...") from exc

        return plaintext
```

Used for:

Workflow:

1. Check skipped keys first: `_try_skipped_message_keys()`
2. If new DH key detected: `_dh_ratchet()` runs.
3. Generate correct message key: `self.CKr, mk = self._kdf_ck(self.CKr)`
4. AES-GCM decrypt: `aesgcm.decrypt()`. If tag invalid: `raise DecryptionError`.

13. State Persistence

```
    def to_dict(self) -> Dict[str, Any]:
        # returns dict containing RK, DHs_priv, DHr_pub, CKs, CKr, PN, Ns, Nr, MKSKIPPED
        ...

    @classmethod
    def from_dict(cls, data: Dict[str, Any]) -> "DoubleRatchet":
        # reconstructs DoubleRatchet instance from the data
        ...
```

Used for:

- `to_dict()`: Stores RK, DH keys, CKs, CKr, Counters, Skipped Keys. Useful for database storage.
- `from_dict()`: Restores session. Without this: Server restart = Conversation lost.

Viva One-Line Summary

If your sir asks: "What is the logic of this file?" Answer:

"This file implements the Signal Double Ratchet algorithm. It takes the shared secret generated by X3DH, creates root and chain keys using HKDF, derives per-message keys using HMAC-SHA256, encrypts messages using AES-GCM, performs Diffie-Hellman ratchets whenever a new public key is received, supports out-of-order message delivery through skipped-key storage, and provides serialization functions so the ratchet state can be stored and restored by the server."

That's the overall logic of the entire code.

Double Ratchet Test Results - Detailed Explanation

Overview

This output shows that the Double Ratchet implementation is functioning correctly across all major scenarios required by a secure messaging application.

Test 1: Basic Exchange + DH Ratchet Trigger

Alice sends the first message to Bob. Bob receives a new DH public key, performs a Diffie-Hellman calculation, and derives a new Root Key, Receiving Chain Key, and Sending Chain Key. This DH ratchet step synchronizes both parties. When Bob replies using his fresh DH key, Alice performs the same ratchet process.

Test 2: Sequential Messages (Symmetric Ratchet)

Messages with counters n=0,1,2,3,4 are encrypted using a chain key that continuously derives new message keys. Every message receives a unique encryption key, providing forward secrecy and preventing future-key prediction.

Test 3: Out-of-Order Delivery

Message n=6 arrives before n=5. The receiver stores skipped message keys while processing later messages. When the delayed message arrives, the stored key is used for successful decryption, matching Signal's behavior.

Test 4: Tampered Ciphertext

AES-GCM authentication detects any modification to ciphertext, nonce, or tag. Verification fails and the message is rejected, protecting against tampering, corruption, and MITM attacks.

Test 5: JSON Serialization

Messages are converted into JSON, transmitted, reconstructed, and decrypted successfully. This confirms compatibility with REST APIs, Flask, Socket.IO, and WebSockets.

Test 6: Session Persistence

Session state is saved and restored using to_dict() and from_dict(). Root keys, chain keys, DH keys, and counters remain intact, allowing conversations to continue after application restart.

Test 7: 200-Message Stress Test

Two hundred encrypted messages are exchanged while periodically triggering DH ratchets. The symmetric ratchet generates fresh keys for every message, while DH ratchets refresh root and chain keys. This demonstrates post-compromise security.

Overall Evaluation

✓ X3DH Session Setup

✓ Double Ratchet

✓ DH Ratchet

✓ Symmetric Ratchet

✓ Message Authentication

✓ Out-of-Order Delivery

✓ Skipped Message Keys

✓ JSON Network Transport

✓ Session Persistence

✓ 200-Message Stress Test

The implementation successfully demonstrates the major security properties of the Signal Protocol: forward secrecy, post-compromise security, authenticated encryption, and reliable message delivery.